0DTE Agent

Automated 0DTE options scanner, scorer, notifier and self-improving dataset engine

.NET 9 · GITHUB ACTIONS · YAHOO FINANCE · GITHUB PAGES

1. Overview

The **0DTE Agent** is an automated intraday scanner and alerting system for same-day-expiry (0DTE) options. During US market hours it continuously watches a small watchlist, looks for option-chain signals across five detection rules, scores each candidate setup on a 0–100 scale, and pushes a notification when a setup clears the quality bar. It also logs a rich, labeled dataset for every candidate — market context, the exact option it would trade, its live greeks and price snapshot, and the outcome the setup produced minutes later.

The agent is designed to be *verifiable, transparent, and self-improving*:

- **Watches** the market and the option chain in real time, with an optional long-running loop or a one-shot mode for cron scheduling.
- **Searches** for setups across five distinct signal types.
- **Scores** each signal by combining market context, news, and an intraday "buy window" evaluation.
- **Notifies** via ntfy.sh push messages (phone push, free, no single-vendor lock-in).
- **Records** every signal — sent or skipped — with market features and a tradeable contract snapshot.
- **Grades** its own alerts after the fact, estimating option P&L so its performance can be audited publicly.
- **Serves** a live status dashboard on GitHub Pages built purely from the recorded dataset.

2. How it works

2.1 Market-aware scanning

The agent only operates during the NYSE regular session (Mon–Fri, 09:30–16:00 US Eastern) and is holiday-aware for 2025–2030 (`MarketHours.cs`). Scheduled runs outside these hours exit cleanly instead of burning compute.

2.2 Data sources

The data layer is pluggable via the `mode` config:

Mode	What it provides	Typical use
`mock`	Synthetic quotes & option chain	Offline development & tests
`yahoo`	Real market data via Yahoo Finance free endpoints: intraday quotes, option chains with bids/asks and greeks, IV, news, previous-day and premarket context	Default production feed (no API key)
`tradier`	Tradier Brokerage API (paper or live)	Broker-backed market data & execution

The Yahoo feed supplies a *reference service* (`YahooReferenceService`) that builds a per-symbol context at scan time: previous-day change %, premarket change %, and the latest headlines. This context feeds the

scorer.

2.3 The scan pipeline

```
Quote ticks + option chain (per symbol)
        ├──►
  ScannerEngine.evaluate(rules)
        ├──► candidate ScanSignals (up to 3 per rule, de-duplicated)
  YahooReferenceService.GetSymbolContext()  ──► news, premarket, prev-day
        ├──► RecommendationScorer.TryScore()  ──► 0–100 score + BuyWindow
  AlertLogStore.AppendAlert()  ──► every candidate is logged, sent or not
        ├──► score ≥ minAlertScore ? NtfyNotifier.Send() : skip
        ├──► BuyWindow actionable ? SendRecommendation() : skip
```

3. Signal rules (scan strategies)

The `ScannerEngine` evaluates up to five rules per scan. Each rule can emit at most **3 signals per scan**, and each specific (rule, strike, side) pair is only flagged once, so the chain is not spammed with repeats of the same setup.

Rule	Signal type produced	Trigger	Direction
Delta sweet spot	Bull call / Bear put	— — quote delta in the |Δ| 0.20–0.50 range	Long/bearish directional
Volume surge	Volume surge	Contract volume more than 2× the chain median	Neutral (momentum)
IV spike	IV spike	Contract IV above 1.5× the ATM IV of the same expiry	Neutral (volatility)
Price level cross	Level cross	Underlying breaks a round price level between scans	Neutral (breakout)
Expiry window	Expiry window	Within 2 hours of expiry (default *off*)	Neutral (theta)

Each quote-based signal carries its **strike, side, and full option quote** (bid, ask, delta, gamma, theta, vega, IV), which is what makes option-level outcome labeling possible.

4. Scoring & buy windows

Every candidate signal is scored 0–100 by `RecommendationScorer`. The score starts at 40 and is adjusted by market context:

Input	Contribution
Premarket change	±3% range × 5 points
Previous-day change	±3% range × 2 points
Recent news	+5 points if any headlines
Buy-window evaluation	window score × 0.25 points

`BuyWindowEvaluator` picks an intraday context from premarket momentum, prior-day trend, news presence, and time until close:

Window	Meaning	Score
Open-range momentum	Premarket move aligns with a directional or volume signal	80
Level breakout	Round price level crossed with momentum	85
Post-news setup	News present, watching for a measured move / fade	55
Pre-close decay	Late in expiry — fast theta / near-the-money focus	50
Trend pullback	Direction intact; buy pullbacks toward prior close	60
No window / flat premarket / market closed	No edge; avoid forced entry	15 / 0

An alert is only *sent* when its final score meets the configured `minAlertScore` (default `55`). Lower setups are still **logged** to the dataset so the threshold can later be learned rather than hand-tuned.

5. Notifications

`NtfyNotifier` posts to a free ntfy.sh topic (or a self-hosted ntfy server) under TLS. A subscription on the phone via the ntfy app receives each alert as a push notification with title, tags, and priority. Supporting fields: optional Bearer token auth for private/self-hosted topics, and ASCII-sanitised headers to keep requests valid.

The default notify message looks like:

```
0DTE · SPY · Bull call
Δ 0.42 · 770 strike · IV 34%
Open-range momentum · Score 78/100
Price $6.40 · 10:32:05
```

When the buy window is actionable, a separate recommendation message also includes the suggested contract symbol.

6. Self-improving dataset layer

This is the heart of the product. Every scan produces three JSONL (JSON-lines) datasets under the configured `dataDir`:

File	Contents	Written by
`alerts.jsonl`	One line per candidate signal, including skips	`AlertLogStore.AppendAlert`
`labels.jsonl`	Outcome grade per alert after it has played out	`OutcomeLabeler.LabelAsync` (`--label`)
`regime.jsonl`	Daily market regime buckets (vol × trend)	`--regime` seed job

6.1 Captured features per alert

Each alert records a fixed, versioned feature set — this is the schema that later machine learning will train on:

Group	Fields

Identity	id, signalTimeUtc, symbol, strategy, direction (+1/−1/0)
Market context	entry spot (underlying), prev-day change %, premarket change %, news count
Scoring	score (0–100), window kind, `sent` (passed threshold or not)
Temporal	days to expiry, hour of day (ET), day of week, regime bucket
Contract snapshot	strike, side, expiry, bid/ask/mid, delta, gamma, theta, vega, implied volatility

6.2 Outcome labeling (option P&L)

`OutcomeLabeler` grades each alert after at least 75 minutes have passed. For the +15 and +60 minute horizons it fetches the underlying's actual intraday path, then reconstructs the trade:

- **Entry:** buy the recorded contract at the recorded *ask*.
- **Exit:** sell at the projected bid at the horizon, computed by *greeks replay* — $\Delta \cdot \Delta S + \frac{1}{2}\Gamma \cdot \Delta S^2 + \Theta \cdot \Delta t$ — minus the recorded half-spread. IV is held flat.
- **Labels recorded:** underlying `dirHit15/60` (direction only, for the directional strategies) *and* projected option return `optionRet15/60Pct`.

> The option P&L is an estimate from the live contract snapshot (real recorded bids/asks and greeks) replayed against the real underlying path. It is conservative (buy ask / sell bid) but is not a real broker fill; live/paper execution is the eventual validation step.

7. Operation modes

Command	Behaviour
`dotnet run`	Long-running loop: scan every `scanIntervalSeconds`, send briefs on a schedule, until Ctrl+C.
`--scan-once`	One full scan pass then exit. Used by the GitHub Actions cron.
`--brief`	Send an on-demand market-context brief (prev-day, premarket, news) and exit.
`--label`	Grade all unlabeled alerts older than the labeling window and exit.
`--regime [--regime-symbol SPY]`	Fetch daily history and rewrite the market-regime history file, then exit.
`--config <path>`	Load a non-default agent.json.

Environment variables override config: `NTFY_TOPIC` and `TRADIER_TOKEN`.

8. Configuration reference (`agent.json`)

```
{
  "ntfyTopic": "0dtes-test-alerts",        // ntfy topic for push notifications
  "symbols": [ "SOFI", "SPY" ],            // watchlist (0DTE expiries)
  "scanIntervalSeconds": 30,               // long-running loop cadence
  "mode": "yahoo",                         // mock | yahoo | tradier
  "paper": true,                           // broker paper mode (tradier)
```

```
  "tradierToken": "",                       // or TRADIER_TOKEN env var
  "enableBrief": true,
  "briefIntervalMinutes": 0,
  "newsPerSymbol": 3,                       // headlines fetched per symbol
  "dataDir": "data",                        // JSONL datasets location
  "minAlertScore": 55,                      // quality gate for sending
  "rules": [                                // per-rule enable/disable
    { "kind": "DeltaSweetSpot", "isEnabled": true  },
    { "kind": "VolumeSurge",    "isEnabled": true  },
    { "kind": "IvSpike",        "isEnabled": true  },
    { "kind": "PriceLevelCross","isEnabled": true  },
    { "kind": "ExpiryWindow",   "isEnabled": false }
  ]
}
```

9. Automation with GitHub Actions

The `.github/workflows/scan.yml` workflow runs the agent in the cloud on a schedule and version-controls the datasets:

Job	Schedule (UTC)	Action
`regime`	12:20 Mon–Fri	Refresh + commit `data/regime.jsonl` before the session
`scan`	every 5 min, 13:30–21:20, Mon–Sun (weekends skipped in-job)	Run `--scan-once`, commit `data/alerts.jsonl` when changed
`label`	21:35 Mon–Fri (after-close)	Run `--label`, commit `data/labels.jsonl` when changed
manual	via `workflow_dispatch`	Trigger any run by hand

The scan job sets `NTFY_TOPIC` from a workflow secret, so notification config stays out of the repository. The 13:30–21:20 UTC window covers the entire US session in both EST and EDT.

10. Live dashboard (GitHub Pages)

The repository ships a static dashboard (`docs/index.html`) deployed on GitHub Pages. It needs no server — it reads the same JSONL datasets directly from the repo. It shows:

- **Status badges:** market open/closed (computed in-browser for ET) and current regime bucket.
- **Summary cards:** alerts logged, sent, labeled, *directional hit%*, and *option hit%*.
- **Recent alerts** table (newest 20): time in ET, symbol, strategy, score, sent/skipped, hit/miss.
- **By-strategy breakdown:** alerts, sent, average score, labeled, directional hit%, option hit% per strategy.

Because every page load recomputes from the raw JSONL, the dashboard doubles as a public, auditable record of the agent's real (including missed) performance.

11. Deployment & architecture

- **Runtime:** .NET 9 console application, nullable-enabled, implicit usings.

- **Packaging:** standalone `Dockerfile` and `fly.toml` for Fly.io deployment.
- **Code layout:** agent code at the repo root; shared/borrowed models & services (`_0dtes_app.*` namespaces) vendored under `src/Models` and `src/Services` so the agent builds as a single self-contained project.
- **Testing:** the shared scanner components carry unit tests in the source `0dtes_app.Tests` project.

12. Roadmap & intent

1. **Data collection phase (now):** run the agent through full trading weeks to accumulate a labeled dataset with option-level outcomes.
2. **Machine learning:** use `alerts.jsonl` features and `labels.jsonl` option-P&L targets to learn which setups actually profit — replacing the hand-tuned score threshold.
3. **Validation & productization:** with a defensible, publicly auditable track record, the agent may evolve into a SaaS offering (signal feeds, dataset licensing, or infrastructure).

13. Honest limitations

- Option P&L labels are **projections** (greeks replay with flat IV between horizons), not broker fills.
- Yahoo market data is free and real-time-ish but **not a broker-grade feed**; fills, spreads and level-2 depth differ.
- The Tradier sandbox (paper) uses **delayed** data — not suitable as the live signal feed.
- Signal quality is currently driven by heuristics; statistical validation requires hundreds of labeled trades, which is exactly why the dataset layer is built in — and why the first weeks are a data-collection phase, not a go-live one.
- Nothing here is financial advice; 0DTE options carry a high risk of total loss.
